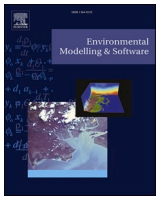
Environmental Modelling and Software 196 (2026) 106753



Contents lists available at ScienceDirect

Environmental Modelling and Software

journal homepage: www.elsevier.com/locate/envsoft





Event-based serverless environmental modeling on the cloud: A pedagogical guide and river case study

Manu Tom [a,*], Cédric H. David [a], Kevin M. Marlis [a], Paul A. Zimdars [a], Quentin Bonassies [b], Jeffrey Wade [a], Arnaud Cerbelaud [a], Matthew Bonnema [a], Tamlin Pavelsky [c], Thomas Huang [a]

[a] Jet Propulsion Laboratory, California Institute of Technology, 91109, Pasadena, California, USA
[b] University of Toulouse, CNRS/Cerfacs/IRD, CECI, 31100, Toulouse, France
[c] University of North Carolina, 27599, Chapel Hill, North Carolina, USA

ARTICLE INFO

Keywords:
Cloud computing
Amazon web services
Lambda
Docker containerization
River discharge

ABSTRACT

Cloud storage of satellite Earth observations motivates moving environmental models closer to the data, while raising concerns about cost and maintainability. We here present a model-agnostic, serverless cloud approach for event-triggered computations that avoids server management on the user-end and leverages containerization for seamless local testing and deployment. Built on Amazon Web Services in proximity to NASA data, our scaffolding uses Lambda and Docker to run models in response to events. We provide a pedagogically structured guide with step-by-step examples for implementing environmental models with minimal infrastructure overhead. A case study of river routing in the Mississippi River Basin demonstrates the approach, simulating 35 years of river discharge. This scaffolding offers a scalable blueprint for deploying environmental models on Amazon Web Services in a data-proximate, serverless, and event-driven fashion.

1. Introduction

The increasing scale, resolution, and frequency of Earth observation data are transforming how and where data are processed and stored. This transformation is particularly acute for recent satellite missions such as the Surface Water and Ocean Topography (SWOT) and the National Aeronautics and Space Administration (NASA) – Indian Space Research Organization (ISRO) Synthetic Aperture Radar (NISAR) which both produce terabytes of data daily (Alsdorf and Lettenmaier, 2003; Alsdorf et al., 2007; Fu et al., 2024; Rosen et al., 2015, 2017; Kellogg et al., 2020). Consequently, their entire data production and archiving pipelines are implemented directly on the computational cloud (Stavros et al., 2020). As a result, many NASA datasets are now archived on the Amazon Web Services (AWS) cloud and served through NASA's *Earthdata* portal.

To efficiently access, process, and ingest these voluminous datasets for Earth system modeling, environmental models (hydrologic, atmospheric, climate, etc.) may benefit from being deployed on the cloud. Cloud deployment can become particularly essential as local infrastructure often lacks sufficient storage and compute capacity to handle data-intensive workflows (Wang et al., 2010; Glenis et al., 2013; Granell et al., 2016; Spichkova et al., 2016). Additionally, cloud-based execution allows models to run closer to the data (Gorelick et al., 2017; Stavros et al., 2020) and hence to access critical data faster, while also minimizing data egress charges. Proximity to data is especially advantageous given the increasing reliance on satellite observations to improve the accuracy of environmental models (e.g., Wongchuig et al., 2024; Zhang et al., 2024). The deployment of models on the cloud could support efficient ingestion of cloud-hosted satellite data for use as boundary conditions, validation, or model-data fusion. Moreover, cloud deployment of environmental models could assist with the implementation of Earth's digital twins (Yang et al., 2013; Hazeleger et al., 2024) which rely on real-time data ingestion, scalable modeling, and seamless integration of diverse data sources.

Cloud computing platforms are now widely used in environmental modeling. The applications include climate modeling, streamflow prediction, fire spread forecasting, groundwater modeling, atmospheric chemistry and real-time weather prediction (Chen et al., 2017, 2018; Qiao et al., 2019; Fraga et al., 2024; Zhuang et al., 2020; Siuta et al., 2016), among numerous others. Perhaps the most widespread cloud computing approach for environmental modeling thus far has been through tailored Virtual Machines (VMs) configured and run as servers

* Corresponding author.
E-mail address: manutom.k@gmail.com (M. Tom).

https://doi.org/10.1016/j.envsoft.2025.106753
Received 14 July 2025; Received in revised form 12 October 2025; Accepted 22 October 2025
Available online 24 October 2025
1364-8152/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

on the cloud. The Elastic Compute Cloud (EC2) allows running such server-based VMs on AWS. EC2 instances function as virtual servers and are often paired with another AWS system called Simple Storage Service (S3) for data management. Large-scale climate models, such as the Community Earth System Model (CESM), were successfully deployed on AWS EC2 servers (Chen et al., 2017) and compared to a local High-Performance Computing (HPC) cluster. In atmospheric chemistry modeling, Zhuang et al. (2020) used the combination of EC2 instances for computational tasks and S3 for storage. Several hydrologic studies also used EC2 servers to deploy their models. Morsy et al. (2018) implemented a system to predict flood impacts on transportation infrastructure. Kim et al. (2021) introduced a cloud-based sensitivity analysis toolbox for the Environmental Fluid Dynamics Code (EFDC) water quality model. EC2 servers are effective and intuitive because they can be manually configured to use the same operating system on which individual environmental models were previously developed or deployed, and they run continuously unless intentionally shut off.

Multiple hydrologic studies have explored cloud-based modeling services that execute on-demand while relying on continuously running servers. David et al. (2014) introduced a Modeling as a Service (MaaS, Roman et al., 2009; Zou et al., 2012; Li et al., 2017) approach using the Cloud Services Innovation Platform (CSIP). CSIP provides Representational State Transfer (REST, Fielding and Taylor, 2002) Application Programming Interface (API) access to hydrologic models such as Engineering Field Handbook Chapter 2 (EFH2). The framework, tailored for a private cloud but compatible with AWS EC2 and S3, also supports a range of hydrologic applications, including erosion models, a watershed model, and a storm runoff model. Tan et al. (2016) implemented a flood emergency response model for the Yangtze River Basin using an Agent-as-a-Service (AaaS) approach deployed on server-based VMs and compatible with both AWS and Alibaba clouds. Similarly, Alcantara et al. (2019) developed a hydrologic MaaS framework that combines the outputs of the Global Flood Awareness System (GloFAS) as inputs to the Routing Application for Parallel Computation of Discharge (RAPID, David et al., 2011a; b, 2016) model and executed as server-based VMs on the Microsoft Cloud. This system builds upon several foundational studies (Swain et al., 2015, 2016; Snow et al., 2016; Qiao et al., 2019; Nelson et al., 2019). Astsatryan et al. (2016) deployed the Soil and Water Assessment Tool (SWAT, Arnold et al., 1998) model as a cloud service using server-based VMs in the OpenNebula cloud. Kurtz et al. (2017) integrated data assimilation into real-time hydrological modeling as server-based VMs compatible with OpenNebula, AWS, and OpenStack clouds. On-demand execution on the cloud has therefore to-date relied entirely on server-based VMs for which there is a need for dedicated server management.

Containerization of software has simplified software deployment by ensuring consistency, portability, and efficient reusability across different environments (Vanegas Ferro et al., 2022); and arguably became a de facto standard in the mid 2010s. Containerized architectures have also been applied in cloud-enabled modeling to support the development of scalable workflows and reduce configuration errors. For example, Taylor et al. (2021) introduced *Basin Futures*, a cloud-based platform designed for exploratory river basin modeling. This system employs the popular Docker solution for containerization with middleware APIs to enable user-guided parameterization and scenario generation. *Basin Futures* integrates rainfall-runoff and water management models within a modular, web-accessible workflow. Likewise, Lawler et al. (2024) used Docker orchestration on the cloud triggered by a standard API in their study of large-scale watershed modeling. However, the studies of Taylor et al. (2021) and Lawler et al. (2024) both use containerized applications with persistent server-based VMs hence requiring manual configuration and scaling when needed.

An often-overlooked benefit of containerization is its ability to create lightweight virtual machines called MicroVMs, which offer the isolation and security of traditional VMs with the speed and efficiency of containers. MicroVMs boot quickly, use minimal resources, and are well-suited for short-lived, serverless workloads. This serverless approach reduces the need for manual server configuration, enables automatic scaling, and supports the variable computational demands of environmental modeling across spatial and temporal scales (Lloyd et al., 2018). It is especially cost-effective for models that run intermittently rather than continuously, eliminating the need to maintain idle infrastructure. Few studies have explored serverless computing for environmental modeling. For example, Zhang et al. (2018) demonstrated a prototype deployment of the Precipitation-Runoff Modeling System (PRMS) hydrologic model on AWS Lambda. Recently, Piciullo et al. (2025) used event-driven Azure Functions for slope stability forecasting, and Markert et al. (2024) applied Google Cloud Run for scalable access to National Water Model forecasts through a containerized serverless API.

The objective of this study is to explore serverless execution of environmental models while focusing on lightweight, event-driven processing to be used specifically when simulations are small or decomposed into smaller tasks. To do so, we use AWS Lambda service which eliminates the need for server provisioning and management. Lambda's pay-per-use model has the potential to reduce idle-time costs and fixed resource requirements compared to earlier hydrologic model deployments on continuously running EC2 instances. The novelty of this work lies in its two-fold contribution: pedagogical and methodological. First, we present an open-source scaffolding for cloud-based numerical modeling on AWS using Lambda and Docker, featuring six modular scaffolds. This aligns with established principles for teaching scientific computing through reproducible, modular learning (Hinsen, 2013; Wilson et al., 2017). Second, the scaffolding is specifically designed for environmental modelers to trigger, deploy, and scale containerized models for relatively small tasks on the cloud in a serverless manner. We demonstrate the scaffolding's capabilities through a case study. Specifically, we conduct multi-decade (1980–2014) river discharge modeling for the Mississippi River basin by deploying the RAPID river routing model (David et al., 2011a, 2011b, 2016) and its pre-processor Reproducible Routing Rituals (RRR, David et al., 2019) on AWS Lambda given our anticipated use of SWOT data together with RAPID.

This paper is arranged as follows. Section 2 discusses the primary components involved in event-triggered serverless cloud deployment of an environmental model illustrated with examples for our river modeling case study. Section 3 presents six pedagogical scaffolds of increasing complexity together leading to the architecture of our serverless cloud scaffolding. The adaptation of a river model to this serverless architecture is then presented (Section 4) before its evaluation (Section 5). We then reflect on the implementation challenges, performance considerations, and broader implications of adopting serverless computing for environmental modeling (Section 6), before providing overall conclusions (Section 7).

2. Methods

Our methodological approach uses the four key components of serverless cloud-based environmental modeling: cloud-native data, non-cloud data, the environmental model, and the serverless cloud implementation itself. Each component is illustrated here with examples from our river modeling case study.

2.1. Cloud-native data from earthdata

Earthdata (https://www.earthdata.nasa.gov/) currently serves as NASA's primary portal to access and visualize Earth science data. *Earthdata* supports research across atmospheric, land, oceanic, and cryospheric domains. The portal provides access to data collected by satellites, aerial surveys, ground-based observations, field measurements, as well as outputs of numerical models. The vast information available in *Earthdata* makes it an essential resource for Earth system modeling and an integral part of our demonstration. Standardized data access from *Earthdata* can be achieved using a variety of approaches.

Here we use the open-source, cloud-compatible Python library called *earthaccess* developed by the National Snow and Ice Data Center.

Among the numerous datasets available in *Earthdata* are outputs from NASA's Global Land Data Assimilation System (GLDAS, Rodell et al., 2004) which serves as an input to our river model. GLDAS produces global fields of land surface states and fluxes and includes three Land Surface Models (LSMs): Variable Infiltration Capacity (VIC, Liang et al., 1994), Noah (Ek et al., 2003), and Catchment Land Surface Model (CLSM, Koster et al., 2000). In our demonstration we use 3-hourly VIC data from GLDAS 2.0 at 1-degree x 1-degree spatial resolution which is available from 1948 to 2014. Other GLDAS model outputs could be used equally although such is beyond the intended scope of this infrastructure design study.

2.2. Non-cloud data from zenodo

The execution of environmental models together with cloud-native data often requires other (smaller) datasets that are available publicly but outside of the cloud. Demonstrating that such data can be safely obtained from (or for) the cloud is therefore valuable in our demonstration. Here, we illustrate our application using *Zenodo* (https://zenodo.org/), an open-access research repository operated by the European Center for Nuclear Research (CERN) that is of common use in the Earth science community.

We leverage an adaptation of the MERIT-Basins hydrography (Lin et al., 2019; Yamazaki et al., 2019;Yang et al., 2021) for global description of river networks together with river reach connectivity and associated simulation parameters produced in a previous study (Collins et al., 2024) and available publicly (Collins et al., 2023) on *Zenodo*. These files do not change with time but are the basis for regular file transformations when new water inputs to rivers are available. In this study, we focus on the Mississippi River Basin which is one of 61 independent regions of the world (Fig. 1). Global implementation would be an incremental step beyond our intended scope.

2.3. Environmental modeling: example of river modeling

Environmental models can differ in their implementation environment (e.g. programming language, operating system) and usually involve both static (i.e. temporally constant) and dynamic (i.e. temporally variable) inputs, the latter often being much more voluminous than the former. A typical cloud deployment would likely attempt to mimic the implementation environment, access large datasets directly from within the cloud, and obtain other static input from outside the cloud before launching.

In our river modeling example, we apply the RAPID model (Figure B.9; David et al., 2011a, 2011b, 2016) together with its pre-processing companion the RRR (David et al., 2019). RAPID is written in Fortran while RRR uses Python; both are routinely developed on a Debian Linux operating system. For clarity, we focus here on the main RRR and RAPID steps involved in model execution as illustrated in Fig. 2, given that implementation details are available in recent literature (e.g. David et al., 2019; Emery et al., 2020; Collins et al., 2024). In our simulations, we focus on a 35-year period (1980–2014). We report hydrographs for the year 2000 as mere verification of implementation – rather than validation against observations – from visual comparison with a previous study (David et al., 2015). We also compare files produced locally and on AWS as an additional verification step.

2.4. Tools supporting serverless cloud deployment on AWS

2.4.1. Amazon S3 for data storage

Amazon S3 (Simple Storage Service) is an object storage system offered by AWS, commonly organized into "buckets." Unlike traditional hierarchical filesystems, S3 uses a flat structure where each object is stored as a key-value pair. The "key" acts like a full path (sometimes using the character "/" as a delimiter), while the "value" holds the actual data. Folders are virtual and inferred from keys (e.g. using "/"), not actual directories. Interactions with S3 can be enabled programmatically. In our river application, we use the *boto3* Python library to interact with our S3 bucket. *Boto3* enables our Lambda functions to query S3 and download input files or upload output files involved in the execution of our environmental models from within the cloud (Fig. 2).

2.4.2. Containerization with Docker, Docker Hub, and the AWS Elastic Container Registry

To streamline the cloud deployment process while facilitating installation of dependencies, we containerize the scaffolds using Docker. We also ensure consistent execution environments across both local ("AWS-like") and cloud (AWS) platforms to enhance portability and reproducibility. We automatically deploy and store Docker images publicly on Docker Hub using GitHub Actions, while also uploading a copy of our images on the AWS Elastic Container Registry (ECR). ECR supports secure and fast retrieval of container images for deployment on AWS services.

2.4.3. AWS Lambda for scalable execution of short model simulations

AWS offers a wide range of computing services, which can make



Fig. 1. The Mississippi Basin is one of 61 independent regions in our global dataset and is used here for demonstration.

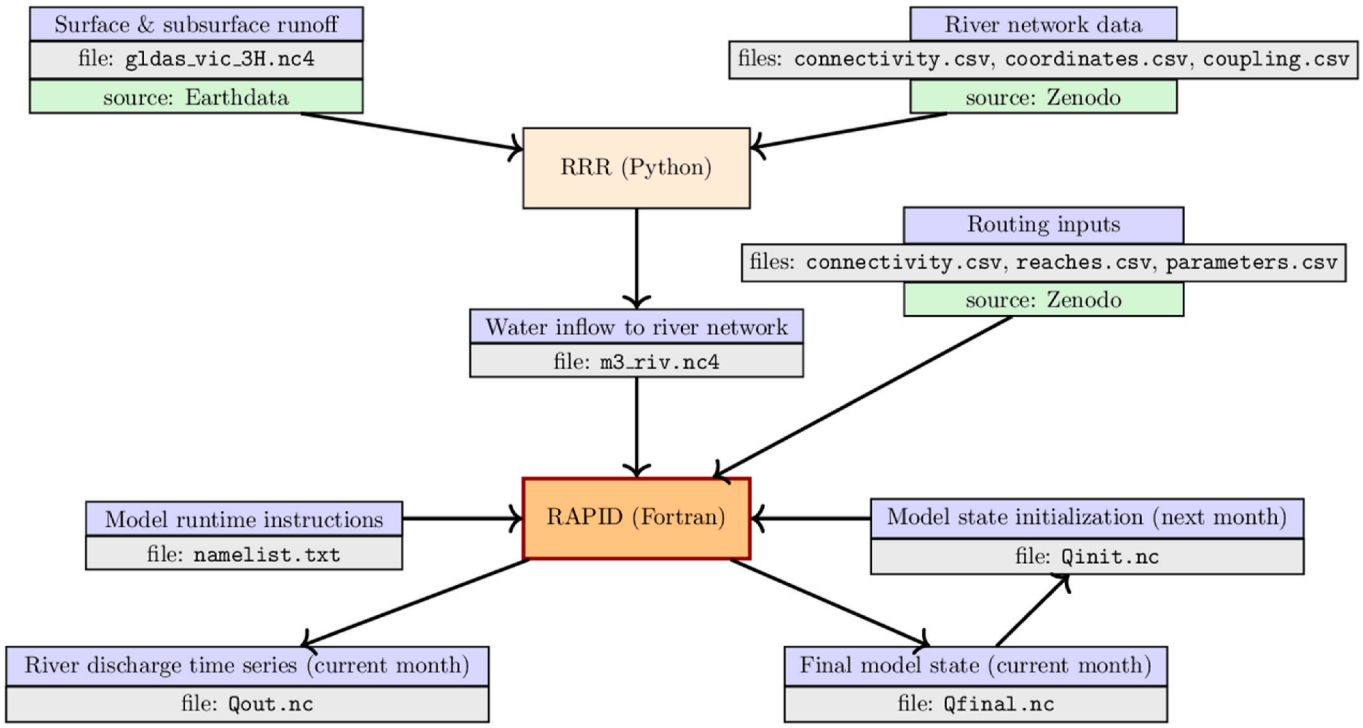


Fig. 2. Workflow diagram illustrating the core hydrologic modeling process using RAPID model and its pre-processing software RRR. Runoff and river network inputs are processed within the cloud to generate runoff inflow files which are then routed through the river network to simulate discharge. Model parameters are obtained from outside of the cloud.

selection challenging for scientific applications. However, most general-purpose workloads rely on three foundational compute systems: EC2, Fargate, and Lambda (Fig. 3). All use VMs but differ in their underlying virtualization models. EC2 uses Nitro-based VMs that provide maximum flexibility and control, but with longer startup times and greater operational overhead. In contrast, Lambda and Fargate both use Firecracker lightweight VMs (MicroVMs), which offer fast startup times and minimal infrastructure management, at the cost of reduced configurability. Firecracker is the basis for AWS's serverless offerings. Although all compute services ultimately run on EC2 bare-metal infrastructure, the underlying virtualization layers-such as Nitro and Firecracker-are abstracted from users. Instead, users interact with higher-level services like EC2, Fargate, and Lambda (Fig. 3). Our literature review indicates that most AWS-based environmental modeling

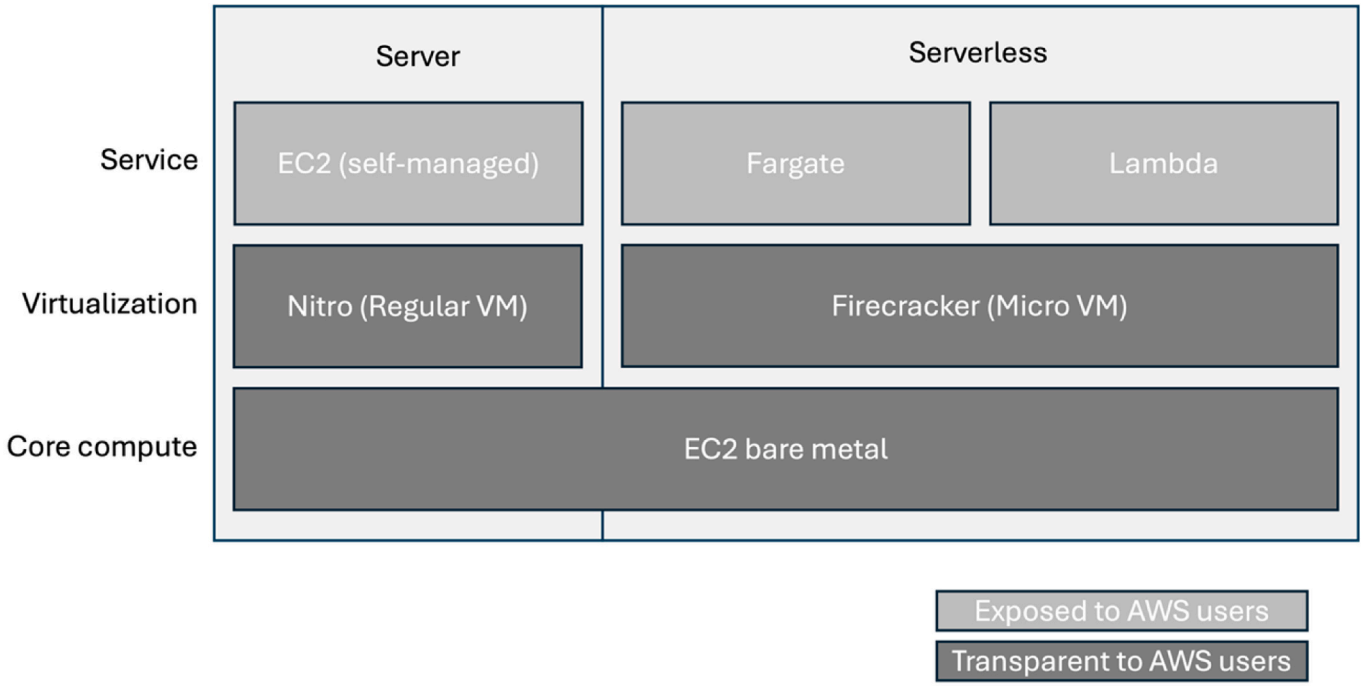


Fig. 3. Primary server-based and serverless compute options on Amazon Web Services.

studies rely on user-managed EC2 instances, which use Nitro atop EC2 bare metal. While one prior study (Zhang et al., 2018) explored one of the two serverless alternatives (AWS Lambda) based on Firecracker, to our knowledge, this study is the first to systematically deploy a containerized numerical model on AWS Lambda. Lambda is designed for short-lived, event-driven tasks with strict time and resource limits, while Fargate supports longer-running workloads with more flexibility in execution and scaling. For event-driven serverless workloads that minimize infrastructure management, three primary options exist: AWS Lambda, AWS Step Function (when paired with Fargate for compute), or AWS Batch (when paired with Fargate for compute and EventBridge for event triggers). AWS Lambda supports containerized workloads directly using Docker images. In contrast, AWS Step Functions and AWS Batch orchestrate or rely on other AWS services-such as Elastic Container Service (ECS) or Elastic Kubernetes Service (EKS)-to run containerized tasks, each with different image handling requirements. In our case, we select Lambda for our intended short and event-driven executions with low operational burden, while noting that Step Function or Batch may be more cost-effective for larger or more complex tasks. We construct custom Docker images for Lambda by applying its Runtime Interface Client (RIC), which facilitates interaction between the Lambda runtime and our own containerized application logic. We also leverage the Lambda Runtime Interface Emulator (RIE) for local "AWS-like" executions. Lambda functions can be triggered by events, such as receiving a JavaScript Object Notation (JSON) message, without the need to manage underlying servers. AWS Lambda offers configurable limits for timeout (3–900 s), memory (128–10,240 MB), and ephemeral storage (512–10,240 MB). Lambda supports two instruction set architectures: *arm64* for Graviton2 processors and *x86_64* for x86-based processors, which determine the underlying Central Processing Unit (CPU) used to run functions. These constraints and choices influence scaffolding runtime, resource allocation, and cost within the Lambda environment.

2.4.4. Simple queue service (SQS)

AWS Lambda can respond to one or multiple events (up to a user-defined batch limit) and launch one or multiple invocations (up to a user-defined concurrency limit) on its own. This model is ideal for embarrassingly parallel workloads that always stays under concurrency and batch limits. However, an additional "queue" system is needed if the number of triggers ever exceeds limits. Additionally, some applications such as time-stepped environmental modeling, require strict ordering of tasks. To support queuing, with both parallel and sequential execution, we use Amazon Simple Queue Service (SQS). SQS is a managed message queue that can buffer and deliver events to Lambda. SQS allows configuration for unordered (standard queue) or ordered (First In, First Out queue) message processing. In our scaffolding, we implemented two queues: a standard queue for parallel execution in the RRR model, and a FIFO queue for time-stepped, sequential execution in the RAPID model. While triggering differs slightly between Lambda alone and SQS/Lambda together (Appendix A), the events themselves are the same.

3. Building blocks for environmental modeling as an AWS Lambda service

Our scaffolding development follows a pedagogical approach to provide an educational structure in which each scaffold builds on the previous one (Fig. 4a). *Scaffolds 0–2* introduce the AWS Lambda environment on a user-defined operating system with basic functionalities

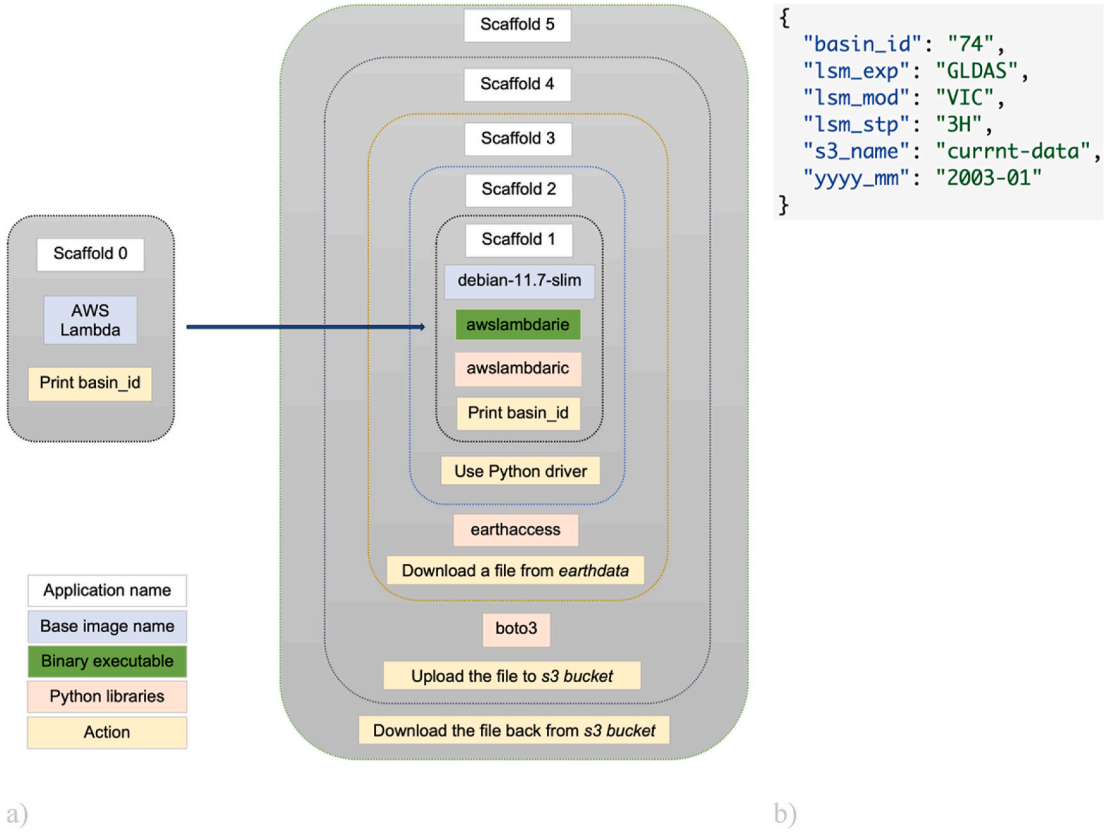


Fig. 4. a) Six pedagogical scaffolds to incrementally add new functionality and cloud-related features. The layered flowchart shows the progression from basic AWS Lambda execution to full data download and upload using an S3 bucket. Together, these scaffolds can serve as building blocks to deploy a numerical model on the AWS cloud in an event-triggered and serverless manner as Lambda service. b) The JSON event that triggers all scaffolds contains six parameters: basin_id (river basin), lsm_exp (Land Data Assimilation System, LDAS), lsm_mod (Land Surface Model, LSM), lsm_stp (simulation frequency), s3_name (S3 bucket name), and yyyy_mm (month and year).

such as printing a message to standard output with and without calling a specific python function defined independently. *Scaffolds 3–5* incorporate more advanced data handling capabilities. Together, these scaffolds provide a template for the different stages of building a cloud-native modeling workflow, from basic Lambda execution to deploying an environmental model on a tailored operating system in a serverless manner. This workflow can be adapted to other models, but it is not ready to support all models as is.

Each scaffold includes a Python script called *lambda_function.py* that includes a *lambda_handler* function with user-defined code that executes in response to events in a serverless manner. Each scaffold also has a *Dockerfile* with instructions to build a Docker image, leveraging the Lambda RIC if using a user-defined operating system. Before migrating to the cloud, we execute each of our scaffolds in an "AWS-like" manner within our local computer using the Lambda RIE. The local environment mirrors the cloud setup and dependencies and allows verification that our scaffolds are compatible with AWS Lambda before deployment. Each scaffold is triggered using an event in JSON format. For uniformity, we use the same event with river-specific parameters (Fig. 4b) to trigger all our generic scaffolds (Scaffolds 0–5) and river modeling scaffolds (Section 4) although some parameters are ignored when not needed by a specific scaffold. The triggering method differs between local and cloud environments while the same JSON event is used (Appendix A). Our software called Combined Utilities for River Routing Nested Together (CURRNT) is shared publicly at https://github.com/c-h-david/currnt. Details of each scaffold are described below.

3.1. Scaffold0: use the Lambda approach

The *Scaffold0* serves as an introductory exercise, focusing on understanding and implementing the Lambda approach, locally and on the AWS cloud. This scaffold retrieves and prints the *basin_id* (Fig. 4b) as a standard output using a basic Lambda function. *Scaffold0* is built upon a public Docker image for AWS Lambda using Python 3.7 (public.ecr.aws/lambda/python:3.7).

3.2. Scaffold1: use a preferred base image

Starting from *Scaffold1*, we replace the default AWS base image used in *Scaffold0* with a custom Debian base image (debian:11.7-slim). This step can be beneficial for complex environmental models that are built on their own specific computational environment. To accommodate the new image, we modify the *Dockerfile* by installing necessary operating system requirements and Python packages, assigning Lambda-specific directories, and integrating the AWS Lambda RIC. We also integrate the RIE and a companion *entry_script.sh*, which work together to facilitate local emulation of the Lambda environment. Despite these modifications, the Lambda function itself remains unchanged.

3.3. Scaffold2: execute a driver

Scaffold2 expands upon the capabilities of *Scaffold1* and executes a separate Python function within the Lambda environment. In this setup, printing the *basin_id* as standard output is achieved with a "driver" Python function that is defined separately and called from the Lambda function. Despite this modification, *Scaffold2* maintains the event triggering and output generation mechanisms of *Scaffold1* and *Scaffold0*.

3.4. Scaffold3: file download from a cloud data archive

Scaffold3 builds upon *Scaffold2* by introducing the capability to download files directly from an online data archive to the execution environment (e.g. locally or on the cloud depending on where it is run). The execution environment in Lambda is ephemeral and locked down for security and performance, so all new files must be written to Lambda's ephemeral storage (mounted to the "*/tmp/*" directory). Here, we provide an example using NASA's *Earthdata* portal to download a sample GLDAS file, which is useful for our river modeling case study. *Dockerfile* modification is needed to incorporate the Python *earthaccess* library for access to *Earthdata*. The convenient but legacy method of storing credentials in a *.netrc* file for use by common downloading tools functions for *Earthdata* in a local AWS-like environment. However, it fails in cloud deployment due to environment constraints unique to Lambda functions. Instead, we use environmental variables to store *Earthdata* credentials. These variables are provided as a command line argument for Docker execution locally or set in the Lambda configuration settings of the AWS console for cloud deployment.

3.5. Scaffold4: upload a file to S3 bucket

Scaffold4 extends *Scaffold3*'s capabilities by enabling users to upload the previously downloaded GLDAS file to a dedicated S3 bucket. The Lambda function orchestrates this upload process leveraging the *boto3* library whose installation is added in *Dockerfile* instructions. The *Scaffold4* driver integrates Python functions for both *Earthdata* download and subsequent upload to the S3 bucket. Unlike non-AWS interfaces like that of *Earthdata*, access to AWS services like S3 relies on credentials that are automatically provided to the Lambda function by AWS itself. This removes the need to manually store or manage AWS credentials in the function environment. In an AWS-like (local) environment, AWS credentials are provided as command line arguments during Docker container execution. When deployed on AWS, a profile-based approach is employed, using a separate configuration file and credential file stored locally for use by the AWS Command Line Interface (CLI).

3.6. Scaffold5: download a file from S3 bucket

Scaffold5 expands upon the features of *Scaffold4* to download the previously uploaded file from the S3 bucket back to the Lambda machine. *Scaffold5* hence allows bidirectional data transfer with S3. As with *Scaffold4*, both *Earthdata* and AWS credentials are required.

4. Adaptation to river data preparation and modeling

We applied the general cloud scaffolding described in Section 3 to a specific environmental modeling suite consisting of RRR for data preprocessing and RAPID for river routing. The resulting serverless architecture, tailored for AWS Lambda, is illustrated in Fig. 5.

4.1. Adaptation of existing environmental model to Lambda

Both RRR and RAPID already maintain dedicated Docker images, automatically deployed to Docker Hub. To enable deployment using AWS Lambda, several key adaptations were necessary.

First, the model workflows were refactored to enable full automation, with all tasks coordinated through a central Python "driver" script each for preprocessing and for routing. These scripts modularize previous ad-hoc workflows into reusable functions and include dependencies for cloud interaction, such as *boto3* for AWS services and *earthaccess* for *Earthdata* downloads.

Second, the model was adapted to respond to AWS Lambda events. This involved the creation of a *lambda_function.py* script to receive and parse incoming event payloads and invoke the appropriate driver functions.

Finally, new Docker images were constructed based on the existing RRR and RAPID images, augmented with Lambda-specific tools and configurations. This included the *aws-lambda-rie* and *entry_script.sh*, which work together to facilitate local emulation of the Lambda environment. The image also installs the AWS RIC Python package (*awslambdaric*) and places the *lambda_function.py* file in the Lambda execution directory *(/var/task/)*, which serves as the entry point when the function is invoked. To minimize runtime latency, all required static

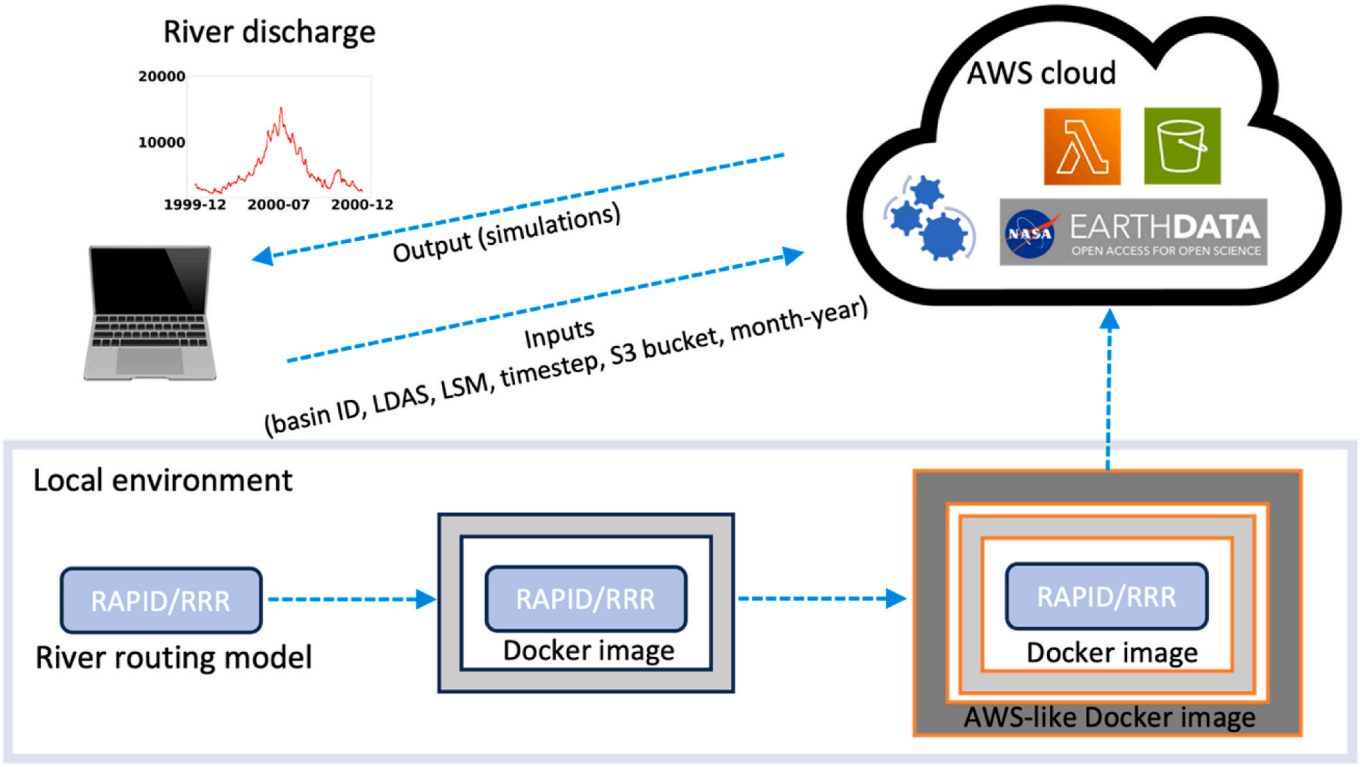


Fig. 5. Serverless environmental modeling triggered by events applied to RAPID and RRR. Before migrating the model to the cloud, the development is done in a local "AWS-like" environment. The system produces river discharge based on six inputs: river basin id, Land Data Assimilation System (LDAS), Land Surface Model (LSM), time step of inputs, name of S3 bucket, and month-year of simulation.

datasets-such as those hosted on *Zenodo*-were embedded directly within the Docker image. While these modifications were necessary for deployment on AWS Lambda, many of them – such as the modular driver logic, containerization, and event response function – can be refactored for other AWS event-driven services or adapted for use with different cloud platforms. In addition, specific tools for formalizing infrastructure as code (e.g. Terraform) could further facilitate deployment across diverse cloud platforms although such is beyond our intended scope.

4.2. Queue driven Lambda orchestration of one-month simulation tasks

To manage execution of environmental models on AWS, we use Amazon SQS to queue simulation tasks and coordinate Lambda invocations. Each task is triggered by a Python-based batch simulator that directly submits event metadata in JSON format (e.g., basin ID, LSM type, target month) to the SQS queue. To ensure each Lambda invocation completes within the 15-min time limit, we configure the SQS batch size to one, so that each invocation handles exactly one task. Concurrent execution limits are also set per model: up to 12 Lambdas run in parallel for RRR, which is embarrassingly parallel, while only one Lambda runs at a time for RAPID to preserve sequential dependency.

Each RRR Lambda instance runs a Python driver that downloads and reformats GLDAS data into the dynamic input to RAPID and saves it to S3. Since GLDAS data is global and reusable across basins, it is downloaded once from *Earthdata*, stored in S3, and reused if present to avoid redundant downloads when processing new basins for a previously downloaded month. Once all RRR jobs finish, RAPID jobs are queued and executed in monthly order, with each Lambda instance using the previous month's final state obtained from S3 to initialize the current month's simulation, before uploading the month-long simulation and final state to S3. While RAPID runs are serialized within a single basin and LSM, simulations across different basins or LSMs can be parallelized. This architecture, illustrated in Fig. 6, enables segmented, month-by-month simulation over multi-decadal periods and supports continuity by managing states through persistent files stored in S3.

4.3. Monitoring and handling runtime challenges

We use AWS CloudWatch to monitor Lambda executions by automatically logging key runtime metrics such as duration, peak memory use, concurrency, and error rates. These logs support debugging by identifying issues like timeouts, failed invocations, or configuration errors. To ensure resilience, we implement error handling for runtime data downloads from *Earthdata* and *Zenodo* (while embedding within the Docker image) addressing risks such as server latency, slow transfer rates, and potential Lambda timeouts.

5. Evaluation of river modeling implementation

5.1. Simulation and verification

We simulate river discharge over the Mississippi River basin using the cloud-native RAPID model (Fig. 7), driven by GLDAS 2.0 runoff produced by the VIC land surface model in a way similar to previous studies (e.g. David et al., 2019; Emery et al., 2020; Collins et al., 2024). The simulation spans 35 years (1980–2014) at a 15-min time step and a 3-hourly input/output resolution, segmented into 840 monthly Lambda jobs (420 jobs each for RRR and RAPID). Unlike David et al. (2015), who used HydroSHEDS hydrography (Lehner et al., 2008) and NLDAS 2.0 runoff for North America (Xia et al., 2012a, 2012b), our setup uses the more recent MERIT-Basins hydrography and global GLDAS, though the VIC and RAPID models are used in both. Our goal here is verifying implementation rather than validating the quality of simulation accuracy.

The spatial patterns of simulated discharge accumulation increase downstream across the Mississippi basin, as expected (Fig. 7a). To verify

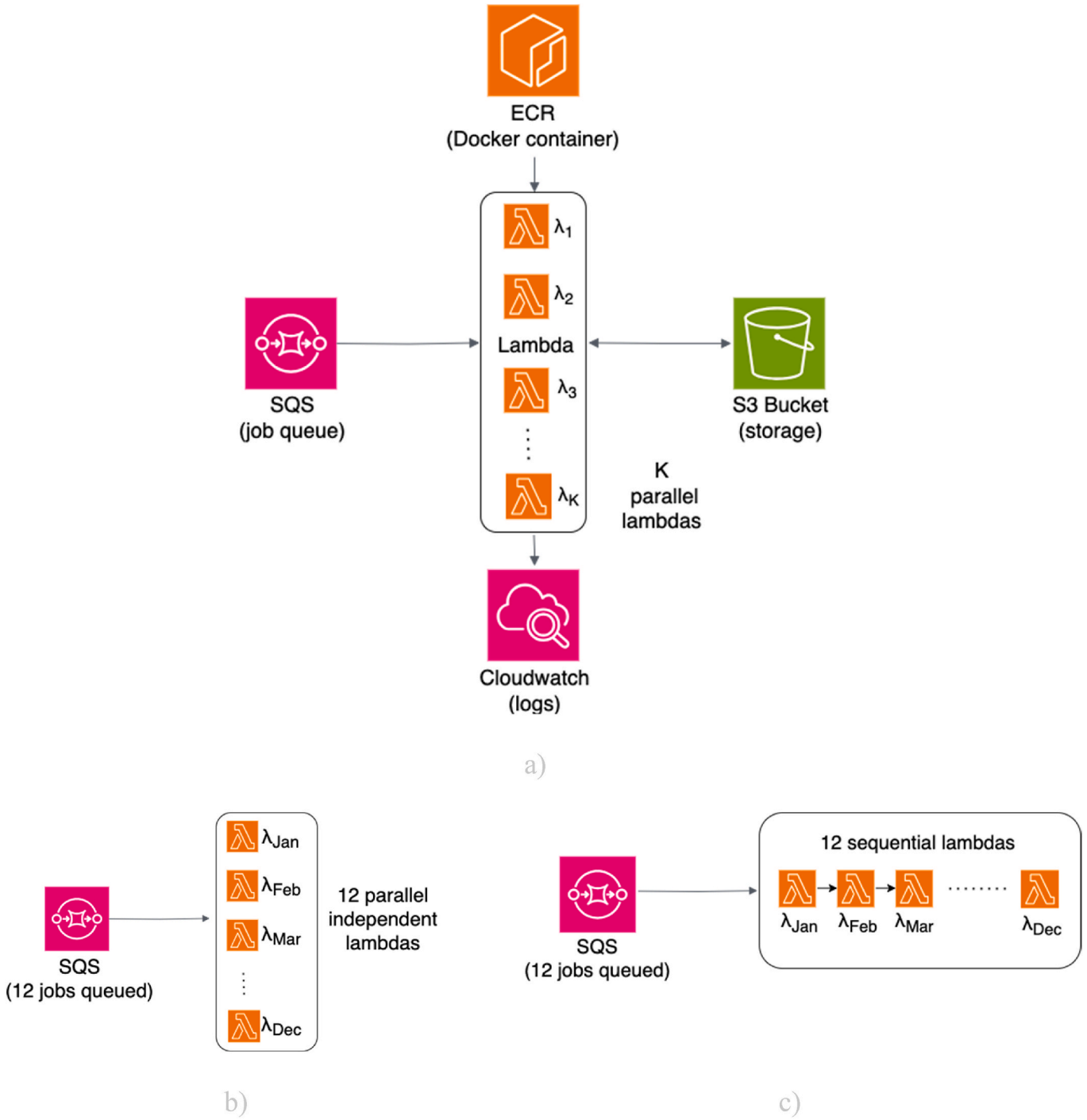
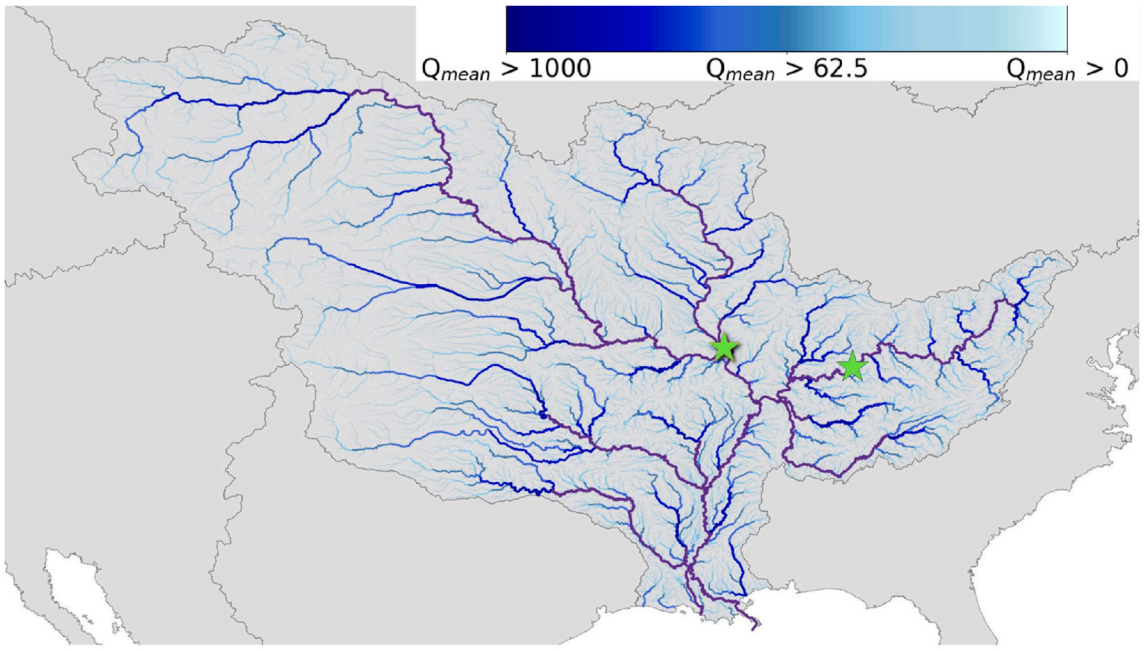


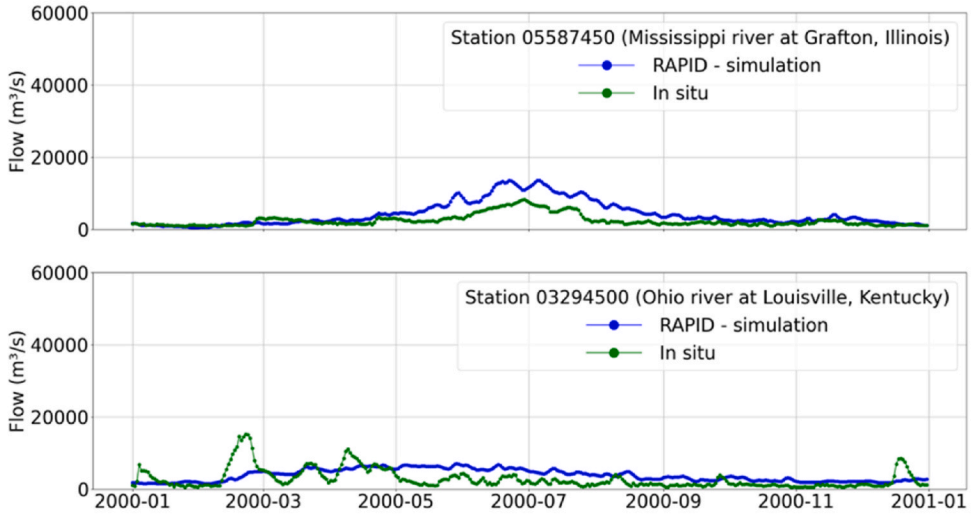
Fig. 6. a) AWS cloud architecture for numerical river modeling (RAPID/RRR). In addition to Lambda, we use S3 bucket for data storage and Cloudwatch for logging and debugging. ECR is used for secure storage of Docker images, and SQS for message queuing. b) RRR inter-lambda workflow (for any year). c) RAPID inter-lambda workflow (for any year). Arrows indicate data flow and execution sequence across the components.

consistency with previous simulations, we compare daily averaged simulated and observed hydrographs at two United States Geological Survey (USGS) gage stations with previously reported results. The simulations reproduce the expected timing and magnitude of seasonal flows (Fig. 7b, compared to Figure A1 in David et al., 2015, not shown here). This confirms that the new inputs yield results consistent with prior studies, as anticipated.

We also verify that outputs from our AWS-based workflow match those from a traditional local environment. For January 2000, Network Common Data Form (NetCDF) files for both RRR and RAPID were byte-identical across cloud and local runs. All data values matched across longitude, latitude, and time dimensions, confirming reproducibility across platforms. Our goal here is verifying implementation rather that validating the quality of simulation accuracy given the breadth of existing studies on calibration for RAPID (e.g. David et al., 2011a; b, 2013, 2016). Consequently, we do not report any quantitative validation metrics.



a)



b)

Fig. 7. a) Mean river discharge (Q_{mean}, 1980–2014) from RAPID simulations (GLDAS 2.0, VIC, 3H) for the Mississippi river basin (MERIT Basins hydrography). Thicker and darker blue indicate higher discharge. Verification points include two USGS gages: Mississippi river at Grafton (left star) and Ohio river at Louisville (right star). b) Hydrographs (year 2000) generated for the two stations can be compared with Figure A1 in David et al. (2015).

5.2. Runtime and cost profiling

To illustrate the order of magnitude of runtime and associated costs, we profile a representative simulation for the year 1980 over our selected basin. Each Lambda function was assigned 1024 MB of memory and 10,240 MB of ephemeral storage on an *arm64* processor.

Twelve RRR jobs were executed in parallel for data processing, completing in under 6 min wall-clock time (i.e. 72 min of cumulative execution time). Followed twelve sequential RAPID jobs of less than 20 s each for river routing, altogether completed in under 4 min of wall-clock

time. Overall RRR consumes 95.2 % of total cumulative execution time while RAPID uses a mere 4.8 % (Fig. 8). Data processing for RRR includes the driver functions for download of LSM files (74.8 %), concatenation of files (3.4 %), generation of RAPID inflow file (2.9 %), and uploads of processed files to S3 (0.9 %). Miscellaneous tasks such as S3 communication and Lambda message parsing contributed marginally (1.1 %). Actual numerical simulation by RAPID constitutes only 3.8 %. Runtime analysis therefore shows that data preparation dominates the total execution time - consistent with broader trends in environmental modeling. The expected lengthy access to key input data further illustrates and justifies the importance of data proximity on the cloud as implemented here. We also exercised the reuse of previously downloaded GLDAS data for the Mississippi river basin. Data reuse for simulations in other basins could notably impact cost saving in a potential global application although such global simulation is beyond the scope of this paper.

Table 1 shows the total cost of $0.079 (USD) for running all RRR and RAPID Lambda functions for this one-year simulation. The bulk of this cost stems from Lambda executions and communication with S3; other services such as ECR, SQS, and Cloudwatch incurred minimal charges. It is important to note that these cost figures are influenced by provider-specific pricing and should be interpreted as illustrative rather than prescriptive. Cloud pricing is a factor in architectural choices, but understanding relative costs is here more important to help us inform future deployment strategies.

Compared to the *arm64* processor, simulations on *x86_64* cost 16.5 % more. Increasing memory to 10,240 MB (from 1024 MB) raised the cost by an order of magnitude (USD 0.678 on *x86_64*). For comparison and at the time of writing, a user-managed EC2 instance with similar specifications (t4g.micro with 1024 MB memory and 10,000 MB storage, *arm64*) costs ~ USD 82/year, while one with 16,384 MB memory and 10,000 MB storage (t3.xlarge, *x86_64*) costs ~ USD 1447/year. The choice between a user-managed EC2, a series of Lambda executions, or other cloud services for our application eventually depends on the anticipated frequency of use. User-managed EC2 may be very well suited for initially running multi-decade simulations, while Lambda could be more appropriate for routine updates with little operational overhead.

Table 1
Execution time and costs for 1 year (1980) simulation (GLDAS, VIC, 3H, Mississippi river basin with over 70000 river reaches) of RAPID and RRR using Lambda and other AWS cloud services.

Scaffold	Runtime	Costs (USD)		
		Lambda	Communication with S3	Total
RRR + RAPID	<10 min	0.058	0.018	0.079
RAPID	<4 min	0.003	0.001	0.006
RRR	<6 min	0.055	0.017	0.075
RRR (reuse downloaded *Earthdata*)	<17 s	0.003	0.001	0.005

6. Lessons learned in cloud-based environmental modeling

This study demonstrates one way in which cloud computing can support environmental modeling by enabling low maintenance, on-demand, and reproducible workflows. Hosting environmental models in a cloud environment provides several advantages including proximity to satellite data archives and scalability. The ability to trigger jobs dynamically and process simulations without maintaining servers reflects a shift toward more accessible and flexible modeling approaches.

However, several challenges arise when adapting environmental workflows to the serverless model. MicroVMs impose unique constraints - such as limited writable directories and more advanced credential mechanisms - that complicate the implementation of traditional modeling pipelines. Additional complexities arise in error handling and debugging, where tracking failures across isolated executions can be cumbersome. To address these, external logic and orchestration are needed to manage dependencies and recover from partial failures, echoing common themes across environmental modeling: managing data, coordinating steps, and maintaining robustness over long time horizons. While these constraints require careful design to manage memory needs, file locations, storage limitations, and execution times, they are necessary tradeoffs for achieving quick, event-driven execution with minimal infrastructure overhead.

This experience also highlights a familiar pattern in environmental workflows: data preparation often dominates total runtime, exceeding the computational cost of the numerical simulations themselves. Such a challenge is not unique to cloud platforms but becomes more visible when execution is broken into discrete, observable steps.

While this study used AWS Lambda for its simplicity and serverless event-driven execution, other cloud services like Fargate-based Batch or Step Function offer alternatives with greater flexibility for compute-intensive or long-running jobs in a serverless way.

Cloud suitability varies with model architecture. Loosely coupled, containerized models can thrive in serverless or hybrid deployments. In contrast, models originally designed for high-performance computing – such as those with tight coupling and message passing parallelism – may face performance limitations in cloud environments without significant adaptation.

Ultimately, this study reflects broader trends in environmental science: the increasing role of cloud-native tools, the ongoing need to tailor deployment strategies to model architecture, and the importance of balancing flexibility, efficiency, and usability in scientific computing.

7. Conclusion

The growing volume of satellite observations and their increasing availability on cloud platforms motivates cloud-native approaches to

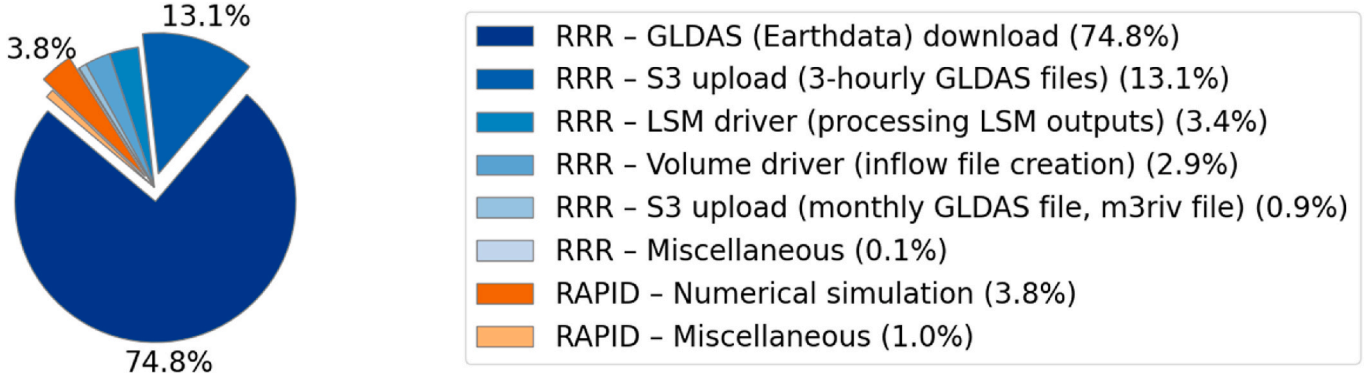


Fig. 8. Execution time profiling for 1 year (1980) simulation (GLDAS, VIC, 3H, Mississippi river basin).

environmental modeling - allowing such data to serve as inputs, validation, or constraint. While cloud-based modeling is becoming more common, our review of existing applications suggests that most rely on persistent virtual machines. These require explicit shutdown, entail ongoing management, and can incur unnecessary costs. Although containerization simplifies deployment, it is often viewed as a tool for traditional servers rather than a lightweight mechanism for serverless computing. In this study, we introduce a novel, open-source scaffolding that enables event-triggered, serverless environmental modeling on the cloud, providing both pedagogical clarity and practical implementation guidance for simulations that are fast or easily decomposed into smaller tasks.

To facilitate this, we adopt AWS for its proximity to NASA data and evaluate AWS Lambda, paired with Docker containers, as a serverless architecture. While alternative services such as AWS Step Functions or AWS Batch may be more appropriate for longer or more complex jobs, our goal was to investigate rapid, on-demand execution with minimal infrastructure overhead.

Informed by challenges encountered while navigating the AWS serverless ecosystem, we developed an open-source, modular scaffolding to guide environmental modelers through step-by-step deployment of cloud-based workflows. We demonstrate the scaffolding using the RAPID river routing model and verify that simulation outputs align with expected discharge patterns.

Runtime profiling shows that model execution is computationally lightweight, while input data preparation - particularly downloading GLDAS runoff - dominates total runtime. This highlights a broader trend in environmental modeling, where data transfer and transformation often outweighs compute time and underscores the importance of data proximity on cloud platforms.

Several technical constraints of AWS Lambda warrant mention. Its 15-min execution limit required partitioning simulations into monthly segments, and its stateless nature demanded custom orchestration for handling dependencies, credentials, and restricted writeable directories. Despite these complexities, the resulting implementation enables scalable, event-driven modeling without the need for persistent servers.

We anticipate that this study will ease our intended use of SWOT data for data assimilation in RAPID and guide future expansion from the Mississippi Basin to global river networks. It is our hope that the scaffolding in this study could serve as a blueprint for adaptation of other containerized numerical models or machine learning workflows in a serverless manner on the AWS cloud. By following its structured progression, environmental scientists might gain insight and practical experience deploying reproducible, modular, and efficient serverless models in the cloud.

CRediT authorship contribution statement

Manu Tom: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Cédric H. David:** Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Kevin M. Marlis:** Validation, Methodology, Conceptualization. **Paul A. Zimdars:** Methodology. **Quentin Bonassies:** Software. **Jeffrey Wade:** Writing – review & editing. **Arnaud Cerbelaud:** Writing – review & editing. **Matthew Bonnema:** Writing – review & editing. **Tamlin Pavelsky:** Writing – review & editing, Funding acquisition. **Thomas Huang:** Writing – review & editing, Project administration, Funding acquisition, Conceptualization.

Software and data availability statement

- Name of software: Combined Utilities for River Routing Nested Together (CURRNT)
  - Developers: Manu Tom, Cédric H. David, and Quentin Bonassies
  - Contact: manu.tom@jpl.nasa.gov, cedric.david@jpl.nasa.gov, and bonassies@cerfacs.fr
- Date first available: December 16, 2024
- Software required: Docker (all other software is installed within Docker image)
- Program language: Python, Shell
- Cost: Free
- Source code and documentation at: https://github.com/c-h-david/currnt
  - The input data for this study is automatically retrieved from the *Earthdata* archive (https://doi.org/10.5067/B6OMXPRI022J) and *Zenodo* (https://doi.org/10.5281/zenodo.8248069) upon execution of the software
  - Sample output dataset (January 1980, February 1980) of this study is publicly available for research purposes at https://doi.org/10.5281/zenodo.14206901
- We follow NASA Open Science guidelines (https://doi.org/10.5281/zenodo.10161526) for our open science practices.

Disclaimer

Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not constitute or imply its endorsement by the United States Government or the Jet Propulsion Laboratory, California Institute of Technology. The cost information contained in this document is of a budgetary and planning nature and is intended for informational purposes only. It does not constitute a commitment on the part of JPL and/or Caltech.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the lead author used ChatGPT 3.5 to improve the readability and language of the first draft manuscript. After using this tool/service, all authors reviewed and edited the manuscript and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration (80NM0018D0004) including grants from the Terrestrial Hydrology (NNH17ZDA001N-THP), Earth Science U.S. Participating Investigator (NNH20ZDA001N-EUSPI), Surface Water and Ocean Topography Science Team (NNH23ZDA001N-SWOTST), and Open Source Software Tools and Libraries (NNH24ZDA001N-OSTFL) Programs. We are grateful to Victoria McDonald (JPL) and Hook Hua (JPL).

Appendix A. Triggering the CURRNT scaffolds

Once the ECR image is deployed to Lambda on the AWS cloud, a scaffold can be triggered by HTTP POST with the *awscurl* command (Table A.2, top). Alternatively, a JSON event (Fig. 4b) can be submitted manually through the AWS console. However, in local AWS-like development environment, the triggering is done using curl command (Table A.2, bottom). Note, for cloud triggers, additional arguments like *region*, *service*, and *profile* are necessary (Table A.2, top).

Table A.2
Commands to trigger the CURRNT scaffolds in AWS and local AWS-like environments.

Environment	Triggering example
AWS	awscurl –region us-west-2 –service execute-api –profile prof-pub -X POST "https://13ghfd7la9.execute-api.us-west-2.amazonaws.com/dev/processes/kt5643/execution/sqs-post" -d { "basin_id": "74″, "lsm_exp": "GLDAS", "lsm_mod": "VIC", "lsm_stp": "3H″, "yyyy_mm": "2000–01″, "s3_name": "currnt-data" }
AWS-like (local)	curl -X POST "http://localhost:9000/2015-03-31/functions/function/invocations" -d { "basin_id": "74″, "lsm_exp": "GLDAS", "lsm_mod": "VIC", "lsm_stp": "3H″, "yyyy_mm": "2001–01″, "s3_name": "currnt-data" }

Appendix B. RAPID parameters

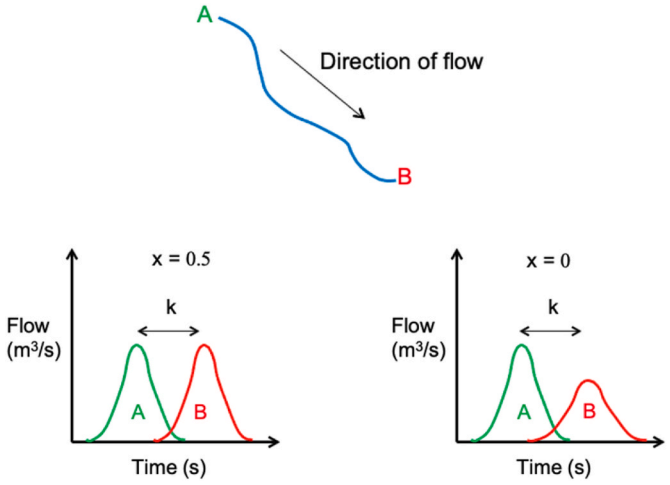


Fig. B.9. Main parameters of RAPID river routing model.

Data availability

Please refer to Software and data availability statement

References

Alcantara, M.A.S., Nelson, J., Shakya, K., Edwards, C., Roberts, W., Krewson, C., Ames, D.P., Jones, N.L., Gutierrez, A., 2019. Hydrologic modeling as a service (HMaaS): a new approach to address hydroinformatic challenges in developing countries. Front. Environ. Sci. 7. https://doi.org/10.3389/fenvs.2019.00158.

Alsdorf, D., Lettenmaier, D., 2003. Tracking fresh water from space. Science 301, 1491–1494. https://doi.org/10.1126/science.1089802.

Alsdorf, D., Rodriguez, E., Lettenmaier, D., 2007. Measuring surface water from space. Rev. Geophys. 45. https://doi.org/10.1029/2006RG000197.

Arnold, J., Srinivasan, R., Muttiah, R., Williams, J., 1998. Large area hydrologic modeling and assessment part I: model development. J. Am. Water Resour. Assoc. 34, 73–89. https://doi.org/10.1111/j.1752-1688.1998.tb05961.x.

Astsatryan, H., Narsisian, W., Asmaryan, S., 2016. SWAT hydrological model as a DaaS cloud service. Earth Sci. Inform. 9, 401–407. https://doi.org/10.1007/s12145016-0254-6.

Chen, X., Huang, X., Jiao, C., Flanner, M.G., Raeker, T., Palen, B., 2017. Running climate model on a commercial cloud computing environment: a case study using

Community Earth system model (CESM) on Amazon AWS. Comput. Geosci. 98, 21–25. https://doi.org/10.1016/j.cageo.2016.09.014.
Chen, C., Chen, D., Yan, Y., Zhang, G., Zhou, Q., Zhou, R., 2018. Integration of Numerical model and Cloud computing. Future Gener. Comput. Syst. 79, 396–407. https://doi.org/10.1016/j.future.2017.06.007.
Collins, E.L., David, C.H., Riggs, R.M., Allen, G., Pavelsky, T.M., Lin, P., Pan, M., Yamazaki, D., Meentemeyer, R.K., Sanchez, G.M., 2023. RRR input and output files corresponding to "Global patterns in river water storage dependent on residence time" (v0.4). Zenodo. https://doi.org/10.5281/zenodo.10013744 [Data set].
Collins, E.L., David, C.H., Riggs, R., Allen, G.H., Lin, P., Pan, M., Yamazaki, D., Meentemeyer, R.K., Sanchez, G.M., 2024. Global patterns in river water storage dependent on residence time. Nat. Geosci. 17, 433–439. https://doi.org/10.1038/s41561-024-01421-5.
David, C., Habets, F., Maidment, D.R., Yang, Z.L., 2011a. RAPID applied to the SIM-France Model. Hydrol. Process. 25, 3412–3425. https://doi.org/10.1002/hyp.8070.
David, C., Maidment, D., Niu, G.Y., Yang, Z.L., Habets, F., Eijkhout, V., 2011b. River network routing on the NHDPlus dataset. J. Hydrometeorol. 12, 913–934. https://doi.org/10.1175/2011JHM1345.1.
David, C., Yang, Z.L., Hong, S., 2013. Regional-scale River flow modeling using off-the-shelf runoff products, thousands of mapped Rivers and hundreds of stream flow gauges. Environ. Model. Software 42, 116–132. https://doi.org/10.1016/j.envsoft.2012.12.011.
David, O., Lloyd, W., Rojas, K., Arabi, M., Geter, F., Ascough II, J.C., Green, T., Leavesley, G., Carlson, J., 2014. Model-As-A-Service (MaaS) using the Cloud Services Innovation platform (CSIP). In: International Congress on Environmental Modelling and Software, San Diego, CA, USA, pp. 15–19. June.
David, C., Famiglietti, J.S., Yang, Z.L., Eijkhout, V., 2015. Enhanced fixed size parallel speedup with the muskingum method using a trans-boundary approach and a large subbasins approximation. Water Resour. Res. 51, 1–25. https://doi.org/10.1002/2014WR016650.
David, C., Famiglietti, J., Yang, Z.L., Habets, F., Maidment, D., 2016. A decade of RAPID-reflections on the development of an open source geoscience code. Earth Space Sci. 3, 226–244. https://doi.org/10.1002/2015EA000142.
David, C., Hobbs, J., Turmon, M., Emery, C., Reager, J., Famiglietti, J., 2019. Analytical propagation of runoff uncertainty into discharge uncertainty through a large River network. Geophys. Res. Lett. 46, 8102–8113. https://doi.org/10.1029/2019GL083342.
Ek, M.B., Mitchell, K.E., Lin, Y., Rogers, E., Grunmann, P., Koren, V., Gayno, G., Tarpley, J.D., 2003. Implementation of Noah Land Surface model advances in the national centers for environmental prediction operational Mesoscale Eta model. J. Geophys. Res. 108, 8851. https://doi.org/10.1029/2002JD003296.
Emery, C., David, C., Andreadis, K., Turmon, M., Reager, J., Hobbs, J., Pan, M., Famiglietti, J., Beighley, E., Rodell, M., 2020. Underlying fundamentals of kalman filtering for River network modeling. J. Hydrometeorol. 21, 453–474. https://doi.org/10.1175/JHM-D-19-0084.1.
Fielding, R.T., Taylor, R.N., 2002. Principled design of the modern web architecture. ACM Trans. Internet Technol. 2, 115–150. https://doi.org/10.1145/514183.514185.
Fraga, E., Cortés, A., Margalef, T., Hernández, P., Carrillo, C., 2024. Cloud based urgent computing for forest fire spread prediction. Environ. Model. Software 177, 106057. https://doi.org/10.1016/j.envsoft.2024.106057.
Fu, L.L., Pavelsky, T., Cretaux, J.F., Morrow, R., Farrar, J.T., Vaze, P., Sengenes, P., Vinogradova-Shiffer, N., Sylvestre-Baron, A., Picot, N., Dibarboure, G., 2024. The surface water and ocean topography Mission: a breakthrough in radar remote sensing of the Ocean and land surface water. Geophys. Res. Lett. 51, e2023GL107652. https://doi.org/10.1029/2023GL107652.
Glenis, V., McGough, A.S., Kutija, V., Kilsby, C., Woodman, S., 2013. Flood modelling for cities using Cloud computing. J. Cloud Comput. 2, 18–27. https://doi.org/10.1186/2192-113X-2-7.
Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., Moore, R., 2017. Google Earth Engine: Planetary-Scale geospatial analysis for everyone. Remote Sens. Environ. 202, 18–27. https://doi.org/10.1016/j.rse.2017.06.031.
Granell, C., Havlik, D., Schade, S., Sabeur, Z., Delaney, C., Pielorz, J., Usländer, T., Mazzetti, P., Schleidt, K., Kobernus, M., Havlik, F., Bodsberg, N., Berre, A., Mon, J., 2016. Future internet technologies for environmental applications. Environ. Model. Software 78, 1–15. https://doi.org/10.1016/j.envsoft.2015.12.015.
Hazeleger, W., Aerts, J.P.M., Bauer, P., Bierkens, M.F.P., Camps-Valls, G., Dekker, M.M., Doblas-Reyes, F.J., Eyring, V., Finkenauer, C., Grundner, A., Hachinger, S., Hall, D. M., Hartmann, T., Iglesias-Suarez, F., Janssens, M., Jones, E.R., Kölling, T., Lees, M., Lhermitte, S., van Nieuwpoort, R.V., Pahker, A.K., Pellicer-Valero, O.J., Pijpers, F.P., Siibak, A., Spitzer, J., Stevens, B., Vasconcelos, V.V., Vossepoel, F.C., 2024. Digital twins of the Earth with and for humans. Commun. Earth Environ. 5, 463. https://doi.org/10.1038/s43247-024-01626-x.
Hinsen, K., 2013. Software development for reproducible research. Comput. Sci. Eng. 15, 60–63. https://doi.org/10.1109/MCSE.2013.91.
Kellogg, K., Hoffman, P., Standley, S., Shaffer, S., Rosen, P., Edelstein, W., Dunn, C., Baker, C., Barela, P., Shen, Y., Guerrero, A.M., Xaypraseuth, P., Sagi, V.R., Sreekantha, C.V., Harinath, N., Kumar, R., Bhan, R., Sarma, C.V.H.S., 2020. NASA-ISRO synthetic aperture radar (NISAR) mission. In: IEEE Aerospace Conference. Big Sky, MT, USA, 07-14 March.
Kim, S., Kwon, Y.S., Pyo, J., Ligaray, M., Min, J.H., Ahn, J.M., Baek, S.S., Cho, K.H., 2021. Developing a cloud-based toolbox for sensitivity analysis of a water quality model. Environ. Model. Software 141, 105068. https://doi.org/10.1016/j.envsoft.2021.105068.
Koster, R.D., Suarez, M.J., Ducharne, A., Stieglitz, M., Kumar, P., 2000. A catchment-based approach to modeling land surface processes in a general circulation model: 1. Model structure. J. Geophys. Res. Atmos. 105, 24809–24822. https://doi.org/10.1029/2000JD900327.
Kurtz, W., Lapin, A., Schilling, O., Tang, Q., Schiller, E., Braun, T., Hunkeler, D., Vereecken, H., Sudicky, E., Kropf, P., Franssen, H.J., Brunner, P., 2017. Integrating hydrological modelling, data assimilation and cloud computing for real-time management of water resources. Environ. Model. Software 93, 418–435. https://doi.org/10.1016/j.envsoft.2017.03.011.
Lawler, S., Zhang, C., Siddiqui, A.R., Lindemer, C., Rosa, D., Lehman, W., Ferreira, C., Di, L., 2024. Leveraging OGC API for cloud-based flood modeling campaigns. Environ. Model. Software 171, 105855. https://doi.org/10.1016/j.envsoft.2023.105855.
Lehner, B., Verdin, K., Jarvis, A., 2008. New global hydrography derived from spaceborne elevation data. Eos Trans 89, 93–94. https://doi.org/10.1029/2008EO100001.
Li, Z., Yang, C., Huang, Q., Liu, K., Sun, M., Xia, J., 2017. Building model as a service to support geosciences. Comput. Environ. Urban Syst. 61, 141–152. https://doi.org/10.1016/j.compenvurbsys.2014.06.004.
Liang, X., Lettenmaier, D.P., Wood, E.F., Burges, S.J., 1994. A simple hydrologically based model of land surface water and energy fluxes for general circulation models. J. Geophys. Res. 99, 14415–14428. https://doi.org/10.1029/94JD00483.
Lin, P., Pan, M., Beck, H., Yang, Y., Yamazaki, D., Frasson, R., David, C., Durand, M., Pavelsky, T., Allen, G., Gleason, C., Wood, E., 2019. Global reconstruction of naturalized River flows at 2.94 million reaches. Water Resour. Res. 55, 6499–6516. https://doi.org/10.1029/2019WR025287.
Lloyd, W., Ramesh, S., Chinthalapati, S., Ly, L., Pallickara, S., 2018. Serverless computing: an investigation of factors influencing microservice performance. In: IEEE International Conference on Cloud Engineering, Orlando, FL, USA, pp. 17–20. April.
Markert, K.N., da Silva, G., Ames, D.P., Maghami, I., Williams, G.P., Nelson, E.J., Halgren, J., Patel, A., Santos, A., Ames, M.J., 2024. Design and implementation of a BigQuery dataset and application programmer interface (API) for the U.S. national water model. Environ. Model. Software 179, 106123. https://doi.org/10.1016/j.envsoft.2024.106123.
Morsy, M.M., Goodall, J.L., O'Neil, G.L., Sadler, J.M., Voce, D., Hassan, G., Huxley, C., 2018. A cloud-based flood warning system for forecasting impacts to transportation infrastructure systems. Environ. Model. Software 107, 231–244. https://doi.org/10.1016/j.envsoft.2018.05.007.
Nelson, E., Pulla, S., Matin, M., Shakya, K., Jones, N., Ames, D., Ellenburg, W., Markert, K., David, C., Zaitchik, B., Gatlin, P., Hales, R., 2019. Enabling stakeholder decision-making with Earth observation and modeling data using tethys platform. Front. Environ. Sci. 7, 6. https://doi.org/10.3389/fenvs.2019.00148.
Piciullo, L., Abraham, M.T., Drøsdal, I.N., Paulsen, E.S., 2025. An operational IoT-based slope stability forecast using a digital twin. Environ. Model. Software 183, 106228. https://doi.org/10.1016/j.envsoft.2024.106228.
Qiao, X., Nelson, E.J., Ames, D.P., Li, Z., David, C.H., Williams, G.P., Roberts, W., Lozano, J.L.S., Edwards, C., Souffront, M., Matin, M.A., 2019. A systems approach to routing global gridded runoff through local high resolution stream networks for flood early warning systems. Environ. Model. Software 120. https://doi.org/10.1016/j.envsoft.2019.104501.
Rodell, M., Houser, P., Jambor, U., Gottschalck, J., Mitchell, K., Meng, C., Arsenault, K., Cosgrove, B., Radakovich, J., Bosilovich, M., Entin, J., Walker, J., Lohmann, D., Toll, D., 2004. The global land data assimilation system. Bull. Am. Meteorol. Soc. 85, 381–394. https://doi.org/10.1175/BAMS-85-3-381.
Roman, D., Schade, S., Berre, A.J., Bodsberg, N.R., Langlois, J., 2009. Model as a service (MaaS). In: Grid Technologies for Geospatial Applications Workshop, Hannover, Germany. June.
Rosen, P.A., Hensley, S., Shaffer, S., Veilleux, L., Chakraborty, M., Misra, T., Bhan, R., Raju Sagi, V., Satish, R., 2015. The NASA-ISRO SAR mission - an international space partnership for science and societal benefit. In: IEEE Radar Conference, Arlington, VA, USA, pp. 10–15. May.
Rosen, P., Hensley, S., Shaffer, S., Edelstein, W., Kim, Y., Kumar, R., Misra, T., Bhan, R., Sagi, R., 2017. The NASA-ISRO SAR (NISAR) mission dual-band radar instrument preliminary design. In: IEEE International Geoscience and Remote Sensing Symposium, Fort Worth, TX, USA, pp. 23–28. July.
Siuta, D., West, G., Modzelewski, H., Schigas, R., Stull, R., 2016. Viability of cloud computing for real-time numerical weather prediction. Weather Forecast. 31, 1985–1996. https://doi.org/10.1175/WAF-D-16-0075.1.
Snow, A.D., Christensen, S.D., Swain, N.R., Nelson, E.J., Ames, D.P., Jones, N.L., Ding, D., Noman, N.S., David, C.H., Pappenberger, F., Zsoter, E., 2016. A high-resolution national-scale hydrologic forecast system from a global ensemble land surface model. J. Am. Water Resour. Assoc. 52, 950–964. https://doi.org/10.1111/1752-1688.12434.
Spichkova, M., Schmidt, H., Yusuf, I., Thomas, I., Androulakis, S., Meyer, G., 2016. Towards modelling and implementation of reliability and usability features for research-oriented cloud computing platforms. In: Maciaszek, L.A., Filipe, J. (Eds.), Evaluation of Novel Approaches to Software Engineering. Springer. Communications in Computer and Information Science, pp. 158–178. https://doi.org/10.1007/978-3-319-56390-9_8.
Stavros, E.N., Oaida, C.M., Hausman, J., Gierach, M.M., 2020. A quantitative framework to inform cloud data system architecture and services requirements based on user needs and expected demand. IEEE Access 8, 138088–138101. https://doi.org/10.1109/ACCESS.2020.3012054.
Swain, N.R., Latu, K., Christensen, S.D., Jones, N.L., Nelson, E.J., Ames, D.P., Williams, G.P., 2015. A review of open source software solutions for developing water resources web applications. Environ. Model. Software 67, 108–117. https://doi.org/10.1016/j.envsoft.2015.01.014.

Swain, N.R., Christensen, S.D., Snow, A.D., Dolder, H., Espinoza- Dávalos, G., Goharian, E., Jones, N.L., Nelson, E.J., Ames, D.P., Burian, S.J., 2016. A new open source platform for lowering the barrier for environmental web app development. Environ. Model. Software 85, 11–26. https://doi.org/10.1016/j.envsoft.2016.08.003.

Tan, X., Di, L., Deng, M., Huang, F., Ye, X., Sha, Z., Sun, Z., Gong, W., Shao, Y., Huang, C., 2016. Agent-as-a-Service-based geospatial service aggregation in the cloud: a case study of flood response. Environ. Model. Software 84, 210–225. https://doi.org/10.1016/j.envsoft.2016.07.001.

Taylor, P., Rahman, J., O'Sullivan, J., Podger, G., Rosello, C., Parashar, A., Sengupta, A., Perraud, J.M., Pollino, C., Coombe, M., 2021. Basin futures, a novel cloud-based system for preliminary river basin modelling and planning. Environ. Model. Software 141, 105049. https://doi.org/10.1016/j.envsoft.2021.105049.

Vanegas Ferro, M., Lee, A., Pritchard, C., Barton, C., Janssen, M., 2022. Containerization for creating reusable model code. Socio. Environ. Syst. Model. 3, 18074. https://doi.org/10.18174/sesmo.18074.

Wang, L., von Laszewski, G., Younge, A., He, X., Kunze, M., Tao, J., Fu, C., 2010. Cloud computing: a perspective study. New Generat. Comput. 28, 137–146. https://doi.org/10.1007/s00354-008-0081-5.

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., Teal, T.K., 2017. Good enough practices in scientific computing. PLoS Comput. Biol. 13, e1005510. https://doi.org/10.1371/journal.pcbi.1005510.

Wongchuig, S., Paiva, R., Siqueira, V., Papa, F., Fleischmann, A., Biancamaria, S., Paris, A., Parrens, M., Bitar, A.A., 2024. Multi-satellite data assimilation for large-scale hydrological-hydrodynamic prediction: proof of concept in the amazon basin. Water Resour. Res. 60, e2024WR037155. https://doi.org/10.1029/2024WR037155.

Xia, Y., Mitchell, K., Ek, M., Sheffield, J., Cosgrove, B., Wood, E., Luo, L., Alonge, C., Wei, H., Meng, J., Livneh, B., Lettenmaier, D., Koren, V., Duan, Q., Mo, K., Fan, Y., Mocko, D., 2012a. Continental-scale water and energy flux analysis and validation for the north American land data assimilation system project phase 2 (NLDAS-2): 1. Intercomparison and application of model products. J. Geophys. Res. 117, D03109. https://doi.org/10.1029/2011JD016048.

Xia, Y., Mitchell, K., Ek, M., Sheffield, J., Cosgrove, B., Wood, E., Luo, L., Alonge, C., Wei, H., Meng, J., Livneh, B., Lettenmaier, D., Koren, V., Duan, Q., Mo, K., Fan, Y., Mocko, D., 2012b. Continental-scale water and energy flux analysis and validation for north American land data assimilation system project phase 2 (NLDAS-2): 2. Validation of model-simulated streamflow. J. Geophys. Res. 117, D03110. https://doi.org/10.1029/2011JD016051.

Yamazaki, D., Ikeshima, D., Sosa, J., Bates, P., Allen, G., Pavelsky, T., 2019. MERIT hydro: a high-resolution global hydrography map based on latest topography dataset. Water Resour. Res. 55, 5053–5073. https://doi.org/10.1029/2019WR024873.

Yang, C., Xu, Y., Nebert, D., 2013. Redefining the possibility of digital Earth and geosciences with spatial cloud computing. Int. J. Digit. Earth 6, 297–312. https://doi.org/10.1080/17538947.2013.769783.

Yang, Y., Pan, M., Lin, P., Beck, H., Zeng, Z., Yamazaki, D., David, C., Lu, H., Yang, K., Hong, Y., Wood, E., 2021. Global reach-level 3-hourly river flood reanalysis (1980-2019). Bull. Am. Meteorol. Soc. 102, 1–49. https://doi.org/10.1175/BAMS-D-20-0057.1.

Zhang, B., Lloyd, W.J., David, O., Leavesley, G., 2018. Going serverless: evaluating the potential of serverless computing for environmental modeling application hosting. In: International Congress on Environmental Modelling and Software, Fort Collins, Colorado, USA, 24–28 June.

Zhang, Y., Chen, X., Stensrud, D.J., Clothiaux, E.E., 2024. Enhancing severe weather prediction with microwave all-sky radiance assimilation: the 10 August 2020 midwest derecho. Geophys. Res. Lett. 51, e2023GL106602. https://doi.org/10.1029/2023GL106602.

Zhuang, J., Jacob, D.J., Lin, H., Lundgren, E.W., Yantosca, R.M., Gaya, J.F., Sulprizio, M. P., Eastham, S.D., 2020. Enabling high-performance cloud computing for Earth science modeling on over a thousand cores: application to the GEOS-chem atmospheric chemistry model. J. Adv. Model. Earth Syst. 12, e2020MS002064. https://doi.org/10.1029/2020MS002064.

Zou, G., Zhang, B., Zheng, J., Li, Y., Ma, J., 2012. MaaS: model as a service in cloud computing and Cyber-I space. In: IEEE International Conference on Computer and Information Technology, Chengdu, China, pp. 27–29. October.